Text Mode

1.

```python
def print_2D(m):
    i = 0
    while i < len(m):
        print_all(m[i])
        i = i + 1

def print_all(m):
    i = 0
    while i < len(m):
        print(m[i], end = " ")
        i = i + 1
    print("\n")


if __name__ == "__main__":
    m = [
        [1, 2, 3],
        [3, 2, 1],
        [9, 1, 1]
    ]
    print_2D(m)
```

```
1 2 3

3 2 1

9 1 1
```

2. วิธีทำคือสร้าง 2D list โดยให้แถวแทนสัปดาห์ และหลักแทนวันในสัปดาห์นั้น จะได้ 2D list ที่มีมิติ 4×7 เข้าถึงได้ผ่านตัวแปร temperature[w][d] โดย w คือสัปดาห์เริ่มสัปดาห์ที่ 1 ที่ $w = 0$ ถึง 3 และ d ลำดับวันในสัปดาห์นั้นเริ่มที่ 0 ถึง 6

จากนั้นใช้ลูปเพื่อประมวลผลข้อมูลที่มีความสัมพันธ์แบบแถวและหลัก

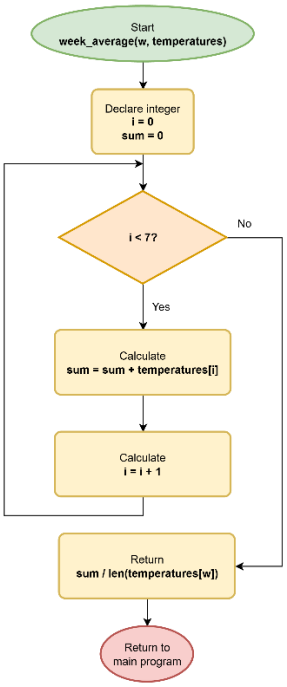
1. ลูปชั้นนอก วนรอบสัปดาห์ (while i < 4) เพื่อสร้างหรือประมวลผลทีละสัปดาห์

2. ลูปชั้นใน วนรอบวัน (while j < 7) เพื่อสุ่มหรือรับค่าอุณหภูมิ 7 ค่าใส่ลงใน List ย่อยของสัปดาห์นั้น (week_temperature)

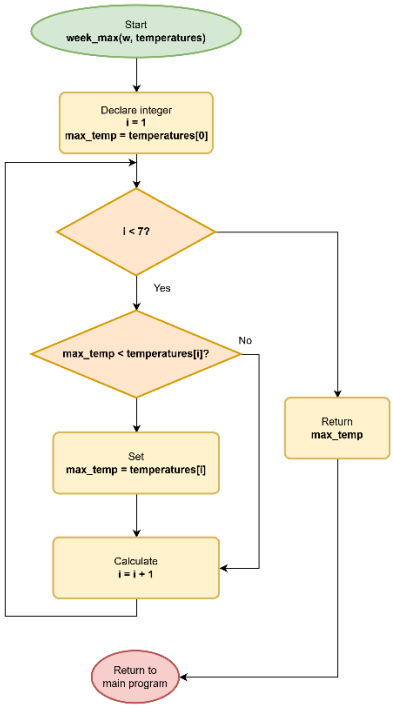
เมื่อจบลูปในแต่ละรอบ จะนำ List ย่อยไปต่อท้าย List หลักด้วยคำสั่ง .append() และต้องรีเซ็ตตัวชี้ j = 0 เพื่อเริ่มสัปดาห์ถัดไป

Flowchart

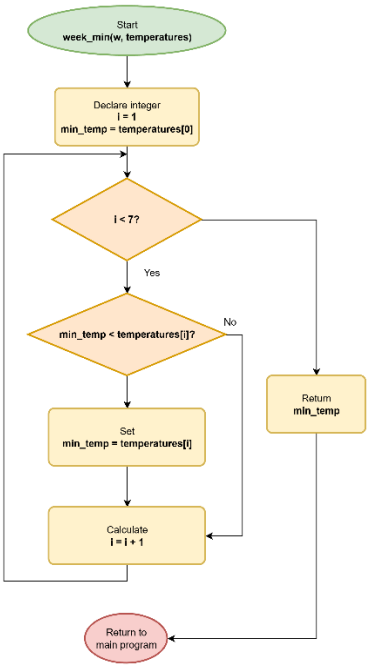
1. ฟังก์ชันแสดงค่าอุณหภูมิเฉลี่ยของสัปดาห์ที่ต้องการ



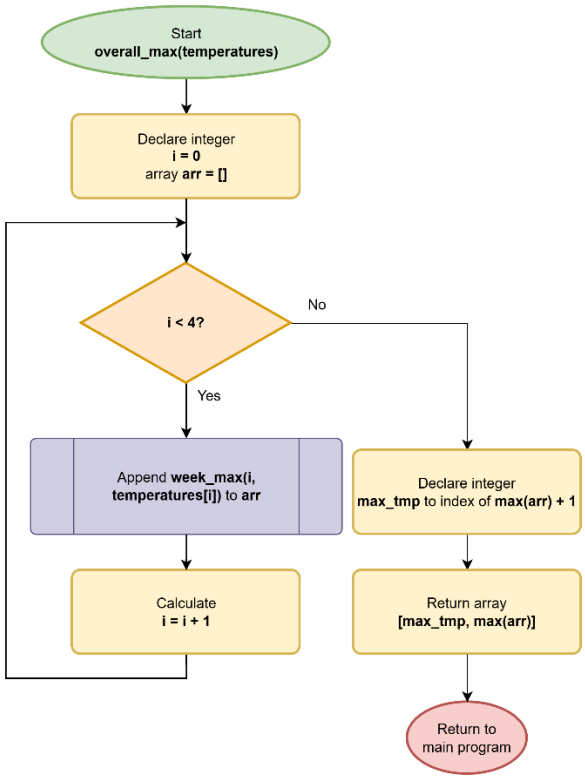
2. ฟังก์ชันแสดงค่าอุณหภูมิสูงสุดของสัปดาห์ที่ต้องการ



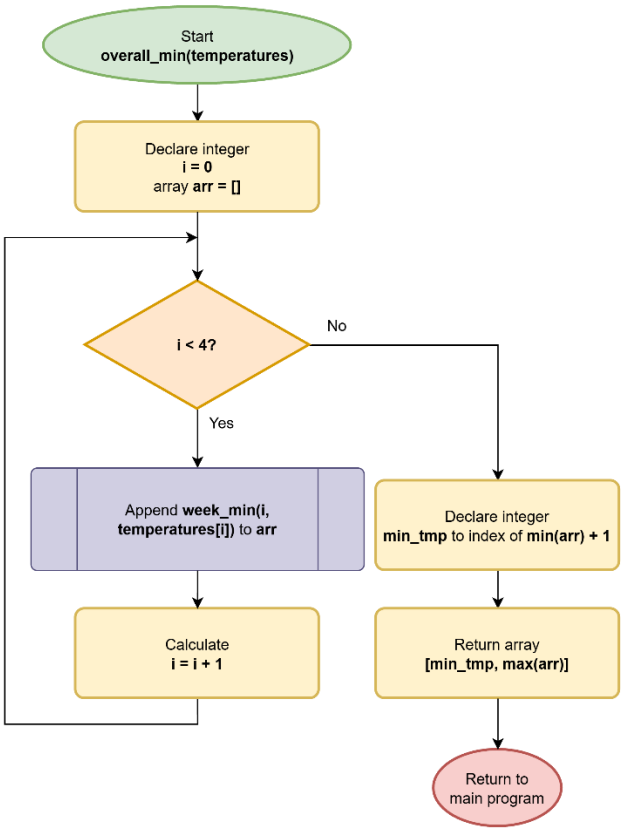
3. ฟังก์ชันแสดงค่าอุณหภูมิต่ำสุดของสัปดาห์ที่ต้องการ



4. ฟังก์ชันแสดงค่าอุณหภูมิสูงสุดของข้อมูลทั้งหมด



5. ฟังกชันแสดงค่าอุณหภูมิต่ำสุดของข้อมูลทั้งหมด



Code และตัวอย่าง Input/Output

```python
import random as rd


temperature = []

def week_average(w, temperatures):
    i = 0
    sum = 0
    while i < 7:
        sum = sum + temperatures[i]
        i = i + 1
    return sum/len(temperature[w])

def week_max(w, temperatures):
    max_temp = temperatures[0]
    i = 1
    while i < 7:
        if max_temp < temperatures[i]:
            max_temp = temperatures[i]
        i = i + 1
    return max_temp

def week_min(w, temperatures):
    min_temp = temperatures[0]
    i = 1
    while i < 7:
        if min_temp > temperatures[i]:
            min_temp = temperatures[i]
        i = i + 1
    return min_temp
def overall_max(temperatures):
    i = 0
    arr = []
    while i < 4:
        arr.append(week_max(i, temperatures[i]))
        i = i + 1
    max_tmp = arr.index(max(arr)) + 1
    return [max_tmp, max(arr)]


def overall_min(temperatures):
    i = 0
    arr = []
    while i < 4:
        arr.append(week_min(i, temperatures[i]))
        i = i + 1
    min_tmp = arr.index(min(arr)) + 1
    return [min_tmp, min(arr)]

```

```
------------------------------------------------------------
Week 1 temerature(C): 22.3, 33.37, 34.37, 30.76, 20.42, 30.84, 21.39
Week 2 temerature(C): 25.18, 22.6, 22.95, 39.6, 35.86, 23.14, 33.37
Week 3 temerature(C): 39.27, 36.82, 23.09, 39.08, 27.44, 24.01, 20.4
Week 4 temerature(C): 35.98, 25.38, 30.08, 30.81, 31.46, 34.01, 33.46
------------------------------------------------------------
Week 1 average temerature: 27.64
Week 2 average temerature: 28.96
Week 3 average temerature: 30.02
Week 4 average temerature: 31.60
------------------------------------------------------------
Week 1 peak temerature: 34.37
Week 2 peak temerature: 39.60
Week 3 peak temerature: 39.27
Week 4 peak temerature: 35.98
------------------------------------------------------------
Week 1 min temerature: 20.42
Week 2 min temerature: 22.60
Week 3 min temerature: 20.40
Week 4 min temerature: 25.38
------------------------------------------------------------
Overall max temerature on week 2 is 39.6
------------------------------------------------------------
Overall min temerature on week 3 20.4
```

3. การหาค่ามากที่สุดของ 2D list ทำได้โดยการใช้ Nested for loop ซึ่งเป็นรูปแบบที่อ่านง่ายกว่า while loop และไม่ต้องพึ่งพาตัวชี้ Index สำหรับการหาค่า Max และ Min

a.) ตั้งค่าเริ่มต้นเป็นสมาชิกตัวแรกสุดเสมอที่ matrix[0][0] เพื่อป้องกันปัญหาในกรณีที่ค่าในเมทริกซ์เป็นค่าลบทั้งหมด แล้วจึง วนลูปเปรียบเทียบทีละตัว หากพบค่าที่มากกว่าจะอัปเดต max_val และหากพบน้อยกว่าจะอัปเดต min_val

สำหรับการหาค่าเฉลี่ย (Average) ใช้ตัวแปรสะสม total เก็บค่านำมาบวกกัน และ count นับจำนวนสมาชิกทั้งหมด คำนวณด้วยสูตร total / count พร้อมดักเงื่อนไขป้องกันกรณีเมทริกซ์ว่าง

b.) ต่อมาการค้นหาสมาชิกใน 2D list ใช้หลักการ linear search คือวนลูปตรวจสอบสมาชิกทุกตัว หากพบค่าที่เท่ากับ target จะ return True ทันทีและออกจากฟังก์ชัน และหากไม่พบจะ return False

c.) เมทริกซ์ทั้งสองจะต้องมีขนาด (Dimension) เท่ากันทุกประการ จึงจะสามารถลูปเอาสมาชิกทั้งสองมาเก็บไว้ใน list ใหม่ได้เมื่อจบลูปแล้วจึง return ผลลัพธ์

ในส่วนของการคูณ จำนวนคอลัมน์ของเมทริกซ์ตัวตั้ง (A) ต้องเท่ากับจำนวนแถวของเมทริกซ์ตัวคูณ (B) เท่านั้น ($Cols_A == Rows_B$) โดยผลลัพธ์ที่ได้จะมีขนาดเป็น $Rows_A \times Cols_B$ มีขั้นตอนวิธีโดยการใช้ Nested Loop 3 ชั้นดังนี้

ลูปชั้นนอกสุด (i): วนรอบแถวของเมทริกซ์ A ($0, 1, 2, \dots, Rows_A - 1$)

ลูปชั้นกลาง (j): วนรอบคอลัมน์ของเมทริกซ์ B ($0, 1, 2, \dots, Cols_B - 1$)

ลูปชั้นในสุด (k): วนตามแนวการคูณแบบ Dot Product ($0 \dots Cols_A - 1$)

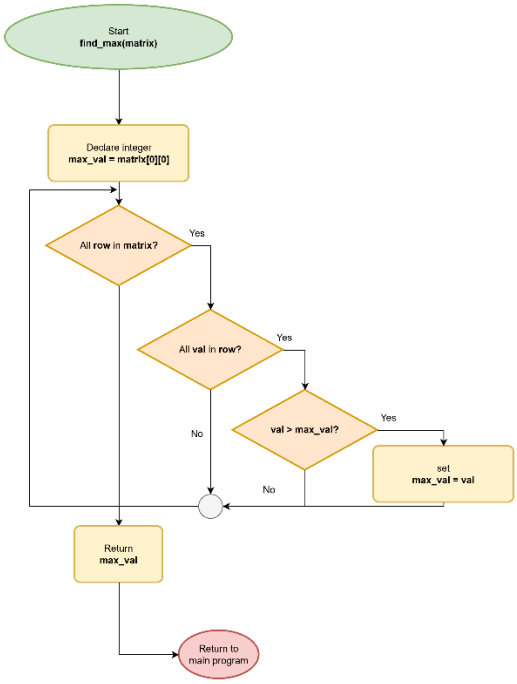
นำสมาชิกแถวที่ i ของ A คูณกับสมาชิกหลักที่ j ของ B แล้วสะสมผลรวม

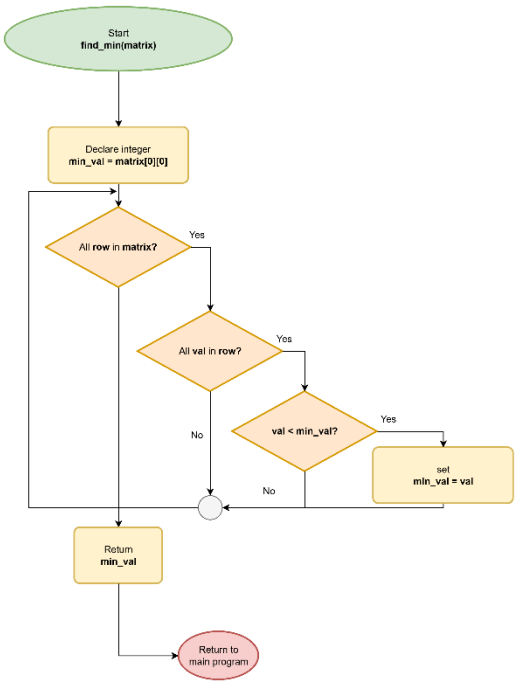
$$\text{cell_sum} = \sum_{k} A\,[i][k] \times B[k][j]$$

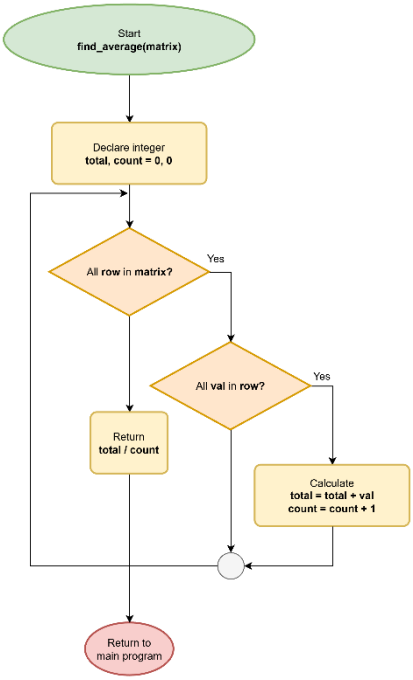
ผลรวม cell_sum ที่ได้ไปสร้างแถวของเมทริกซ์ผลลัพธ์

Flowchart

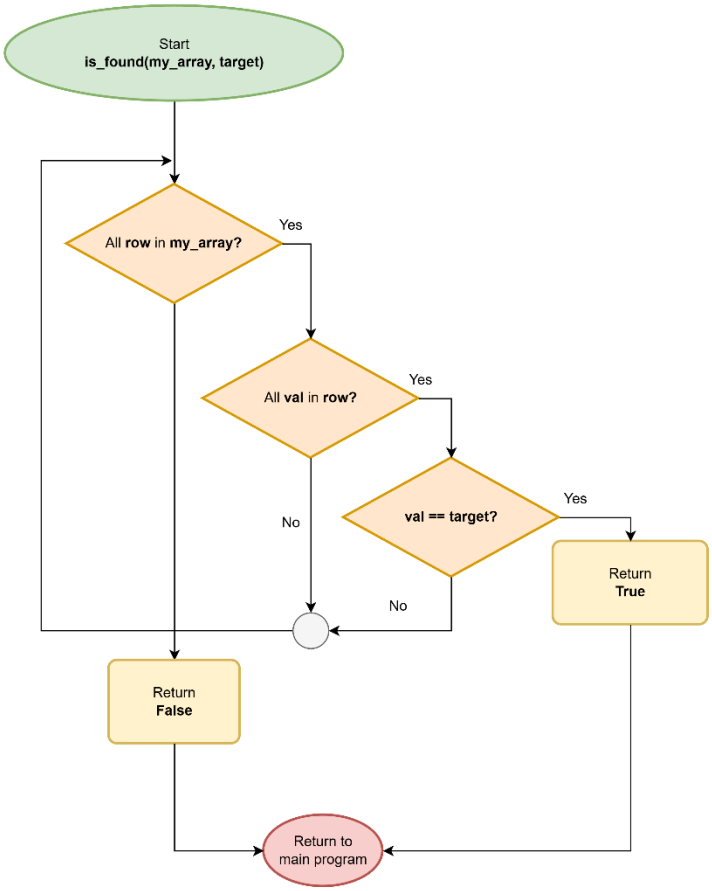
a.) max/min/average of 2D Array



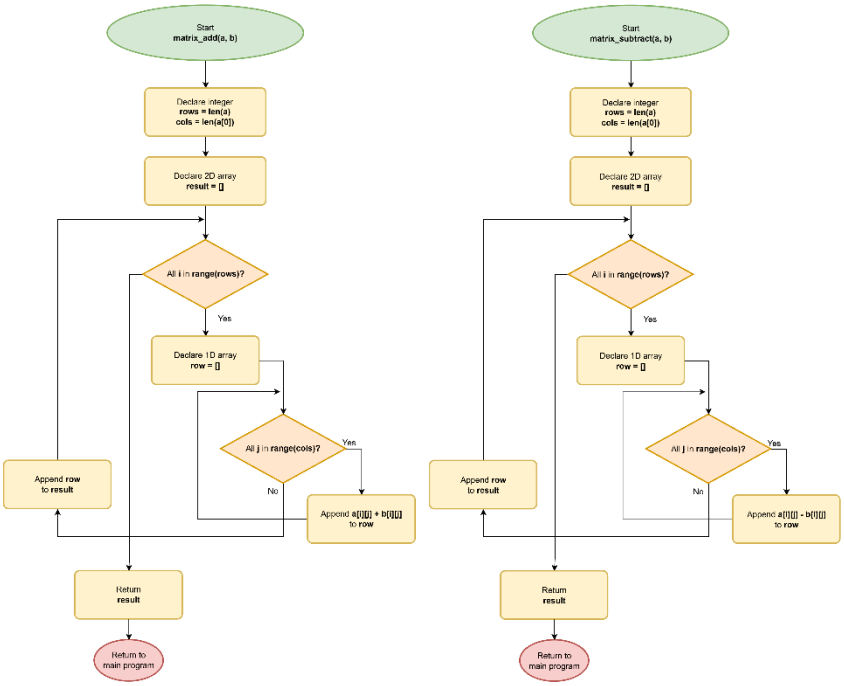


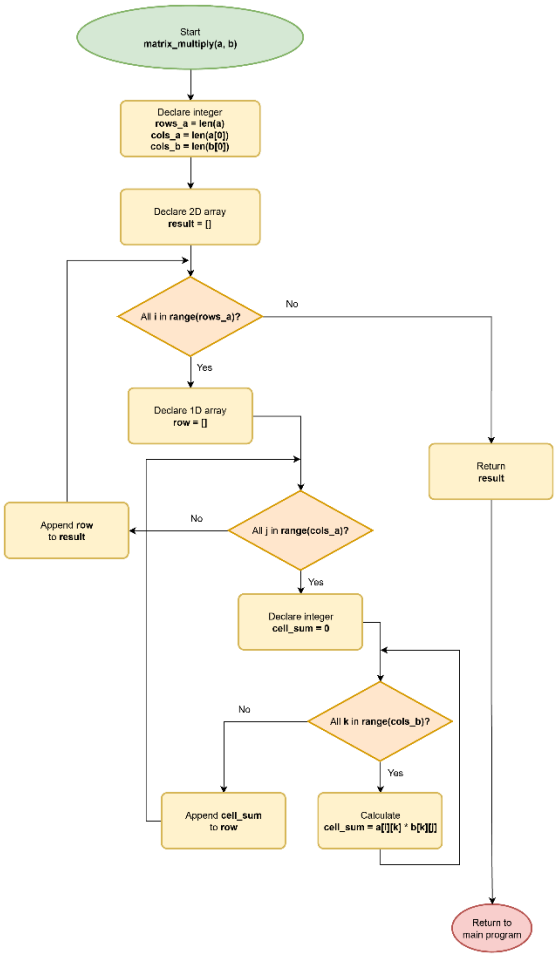


b.) check whether a value is in a 2D array (return boolean)



c.) matrix add, subtract, multiplication





Code และตัวอย่าง Input/Output

```python
#Part A: Max, Min, and Average


def find_max(matrix):
    max_val = matrix[0][0]
    for row in matrix:
        for val in row:
            if val > max_val:
                max_val = val
    return max_val


def find_min(matrix):
    min_val = matrix[0][0]
    for row in matrix:
        for val in row:
            if val < min_val:
                min_val = val
    return min_val


def find_average(matrix):
    total = 0
    count = 0
    for row in matrix:
        for val in row:
            total += val
            count += 1
    return total / count if count > 0 else 0
#Value Search


def is_found(my_array, target):
    for row in my_array:
        for val in row:
            if val == target:
                return True
    return False


#Part C: Matrix Operations
def matrix_add(a, b):
    rows = len(a)
    cols = len(a[0])
    result = []
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(a[i][j] + b[i][j])
        result.append(row)
    return result


def matrix_subtract(a, b):
    rows = len(a)
    cols = len(a[0])
    result = []
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(a[i][j] - b[i][j])
        result.append(row)
    return result
```

```python
def matrix_multiply(a, b):
    rows_a = len(a)
    cols_a = len(a[0])
    cols_b = len(b[0])

    result = []
    for i in range(rows_a):
        row = []
        for j in range(cols_b):
            cell_sum = 0
            for k in range(cols_a):
                cell_sum += a[i][k] * b[k][j]
            row.append(cell_sum)
        result.append(row)
    return result
```

```
Enter number of rows for Matrix A: 1
Enter number of columns for Matrix A: 3
Enter the elements for Matrix A (space-separated per row):
Row 1: 1 2 3

--- Matrix A Statistics ---
Maximum: 3
Minimum: 1
Average: 2.00

Enter a value to search in Matrix A: 5
Is 5 found in Matrix A?: False

--- Enter Matrix B ---
Enter number of rows for Matrix B: 3
Enter number of columns for Matrix B: 1
Enter the elements for Matrix B (space-separated per row):
Row 1: 5
Row 2: 6
Row 3: 7

--- Matrix Operations ---

Addition and Subtraction require Matrix A and B to have matching dimensions.

Matrix Multiplication (A x B):
[38]
```

Graphic Mode

4. การทำเกม Tic tac toe อย่างง่ายนั้นมีโครงสร้างพอสังเขป 4 ส่วนด้วยกันได้แก่

1. ตัวแปรที่ใช้เก็บสถานะของเกม (Global variables)

    - board: 2D array ขนาด 3 x 3 โดยเริ่มต้นเก็บค่าเป็นช่องว่างภายหลังจะเก็บเป็น ‘X’ หรือ ‘O’

    - current_player: ระบุตาของผู้เล่น มักจะเริ่มต้นที่ ‘X’

    - game_over และ winner: ใช้ควบคุมการหยุดเล่นและบันทึกผู้ชนะหรือเกมเสมอ

2. รอบการทำงานของ Processing:

    - setup(): กำหนดขนาดหน้าต่าง 600 x 600 pixels

    - draw(): ใช้วาดพื้นหลัง เส้นกระดาน สัญลักษณ์ต่างๆ

3. User-defined function ที่ใช้ในการแสดงผล

    - draw_grid(): ขีดตาราง 2 เส้นแนวตั้ง และ 2 เส้นในแนวนอน ช่องละ 200 x 200 pixels

    - draw_marks(): ใช้แปลงตำแหน่งช่องแต่ละช่องเป็นพิกัดของกึ่งกลางเพื่อวาดเครื่องหมายกากบาทและวงกลม

    - display_status(): แสดงกล่องข้อความเพื่อบอกผลลัพทธ์ของเกม

4. ระบบควบคุมเกม

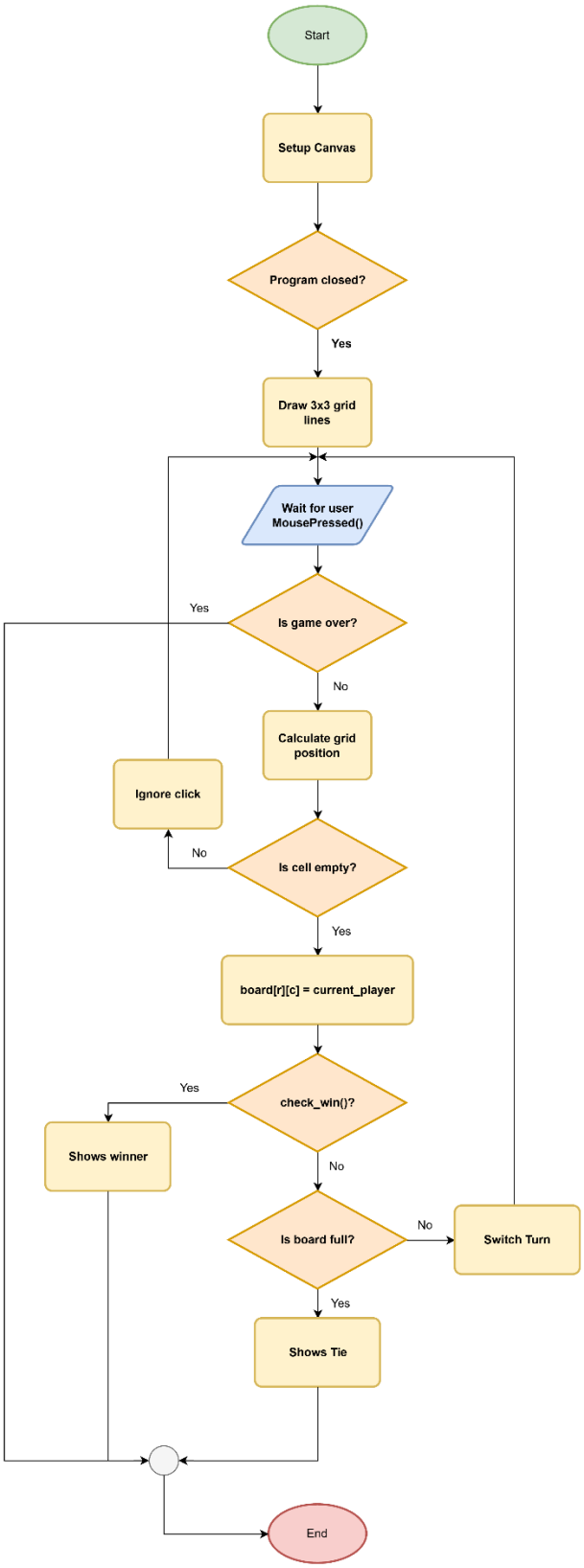
    - mousPressed(): เป็น Built-in function ใน Processing ใช้เพื่อลงเครื่องหมายหรือกดเริ่มเกมใหม่

    - check_win(): ตรวจสอบ winning combination ของเกมทั้งแต้มในแนวนอน แนวตั้ง และแนวทแยง

    - is_full(): ใช้เพื่อตรวจผลเสมอ เมื่อไม่มีช่องว่างสำหรับลงสัญลักษณ์

    - reset_game(): เคลียร์ตารางและตั้งค่าตัวแปรใหม่ เพื่อเริ่มเกมใหม่ทั้งหมด

Flowchart


Start
Setup Canvas
Program closed?
Yes
Draw 3x3 grid lines
Wait for user MousePressed()
Is game over?
Yes
No
Calculate grid position
Ignore click
Is cell empty?
No
Yes
board[r][c] = current_player
check_win()?
Yes
Shows winner
No
Is board full?
No
Switch Turn
Yes
Shows Tie
End

Code และตัวอย่าง Input/Output

```
board = [
    ['', '', ''],
    ['', '', ''],
    ['', '', '']
]
current_player = 'X'
game_over = False
winner = None

def setup():
    size(600, 600)
    strokeWeight(6)
    textAlign(CENTER, CENTER)

def draw():
    background(245)
    draw_grid()
    draw_marks()
    display_status()

def draw_grid():
    stroke(80)
    strokeCap(ROUND)

    line(200, 20, 200, 580)
    line(400, 20, 400, 580)

    line(20, 200, 580, 200)
    line(20, 400, 580, 400)

def draw_marks():
    for r in range(3):
        for c in range(3):
            mark = board[r][c]
            x = c * 200 + 100
            y = r * 200 + 100

            if mark == 'X':
                stroke(220, 60, 60)
                offset = 45
                line(x - offset, y - offset, x + offset, y + offset)
                line(x + offset, y - offset, x - offset, y + offset)
            elif mark == 'O':
                stroke(50, 120, 220)
                noFill()
                ellipse(x, y, 110, 110)

def mousePressed():
    global current_player, game_over, winner


    if game_over:
        reset_game()
        return


    c = mouseX // 200
    r = mouseY // 200

    if 0 <= r < 3 and 0 <= c < 3:
        if board[r][c] == '':
            board[r][c] = current_player

            if check_win(current_player):
                game_over = True
                winner = current_player
            elif is_full():
                game_over = True
                winner = 'Tie'
            else:
                current_player = 'O' if current_player == 'X' else 'X'
```

```
def check_win(player):

    for i in range(3):
        if board[i][0] == board[i][1] == board[i][2] == player:
            return True
        if board[0][i] == board[1][i] == board[2][i] == player:
            return True

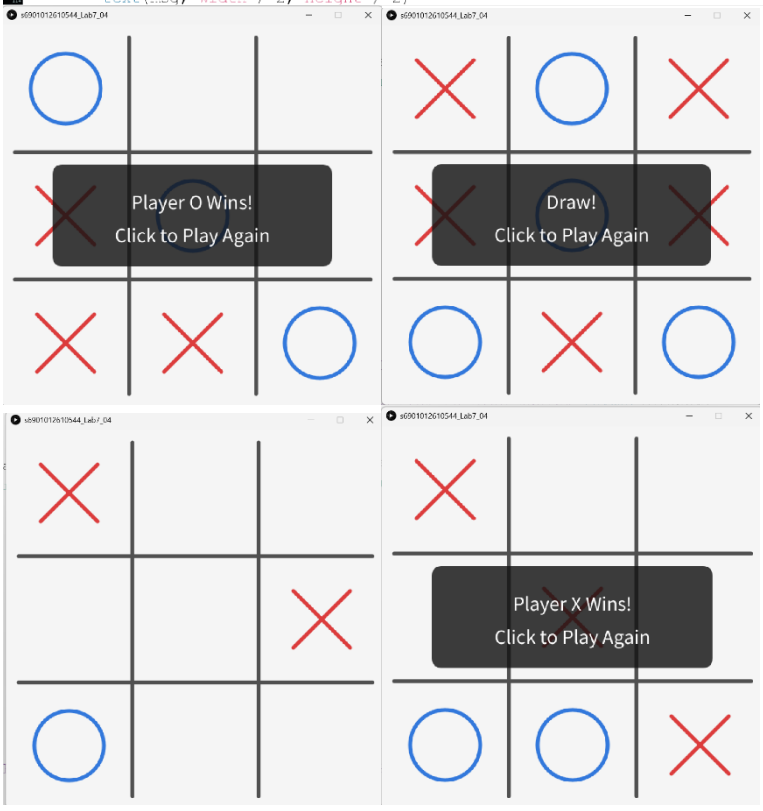

    if board[0][0] == board[1][1] == board[2][2] == player:
        return True
    if board[0][2] == board[1][1] == board[2][0] == player:
        return True

    return False
def is_full():
    for row in board:
        if '' in row:
            return False
    return True

def reset_game():
    global board, current_player, game_over, winner
    board = [['', '', ''] for _ in range(3)]
    current_player = 'X'
    game_over = False
    winner = None

def display_status():
    if game_over:
        fill(20, 20, 20, 210)
        noStroke()
        rect(80, 220, 440, 160, 16)

        fill(255)
        textSize(32)
        if winner == 'Tie':
            msg = "Draw!\nClick to Play Again"
        else:
            msg = "Player {} Wins!\nClick to Play Again".format(winner)
        text(msg, width / 2, height / 2)
```



5.โครงสร้างหลักของโค้ดเกม Minesweeper นี้แบ่งออกเป็นการกำหนดค่าสถานะ การคำนวณตำแหน่งระเบิด การแสดงผล และการจัดการการเล่น:

1. ค่าคงที่และตัวแปรสถานะ

   - กำหนดขนาดกระดานผ่าน ROWS = 10, COLS = 10, ขนาดช่อง CELL_SIZE = 40 และจำนวนระเบิด NUM_MINES = 10

   - ใช้ตาราง 2 มิติ จัดการสถานะช่องต่างๆ ได้แก่ mines (ตำแหน่งระเบิด), revealed (ช่องที่เปิดแล้ว), flagged (ช่องที่ปักธง), และ neighbor_counts (จำนวนระเบิดรอบช่องนั้น)

   -first_click: ควบคุมไม่ให้ผู้เล่นเหยียบระเบิดตั้งแต่คลิกแรก

2. วงรอบการทำงานและการจัดวางหน้าจอ

   -settings() และ setup(): คำนวณขนาดหน้าต่างแบบ dynamic ให้พอดีกับตารางรวมกับแถบหัวด้านบน (HEADER_HEIGHT = 60)

   -draw(): วาดพื้นหลัง แถบข้อมูลด้านบน draw_header() และตารางเกม draw_board()

3. การสุ่มระเบิดและการคำนวณตัวเลข (Mine Placement)

   -place_mines(first_r, first_c): ทำงานเมื่อคลิกครั้งแรก โดยสุ่มวางระเบิดจากช่องทั้งหมดที่ไม่ใช่ช่อง (first_r, first_c)

   -วนลูปคำนวณล่วงหน้า (Precompute) หน่วงนับจำนวนระเบิดใน 8 ทิศทางรอบตัวแล้วบันทึกเก็บไว้ใน neighbor_counts

4. การแสดงผลกราฟิก (Display)

   -draw_header(): แสดงจำนวนระเบิดที่เหลือ (หักลบกับจำนวนธงที่ปัก), ขนาดกระดาน, และข้อความแจ้งเตือนสถานะเมื่อชนะหรือแพ้

   -draw_board():

      - ช่องที่ยังไม่เปิด: วาดพื้นหลังสีเทาเข้ม พร้อมรูปธงสีแดงหากถูกปักธงไว้

      - ช่องที่เปิดแล้ว: หากเป็นระเบิดจะวาดวงกลมสีแดง หากไม่ใช่ระเบิดจะแสดงตัวเลขตามสีเฉพาะของแต่ละหมายเลข (1–8)

5. ระบบควบคุมและตรรกะเกม (Logic & Input)

1. mousePressed():

- หากเกมจบแล้ว การคลิกจะสั่ง reset_game() เพื่อเริ่มใหม่

- แปลงพิกัดเมาส์เป็นแถวและคอลัมน์ โดยหักลบระยะ HEADER_HEIGHT ออกก่อน

- คลิกขวา: สลับสถานะปักธง/ถอดธง (flagged)

- คลิกซ้าย: เปิดช่องผ่าน reveal_cell() และตรวจเงื่อนไขชนะด้วย check_win()

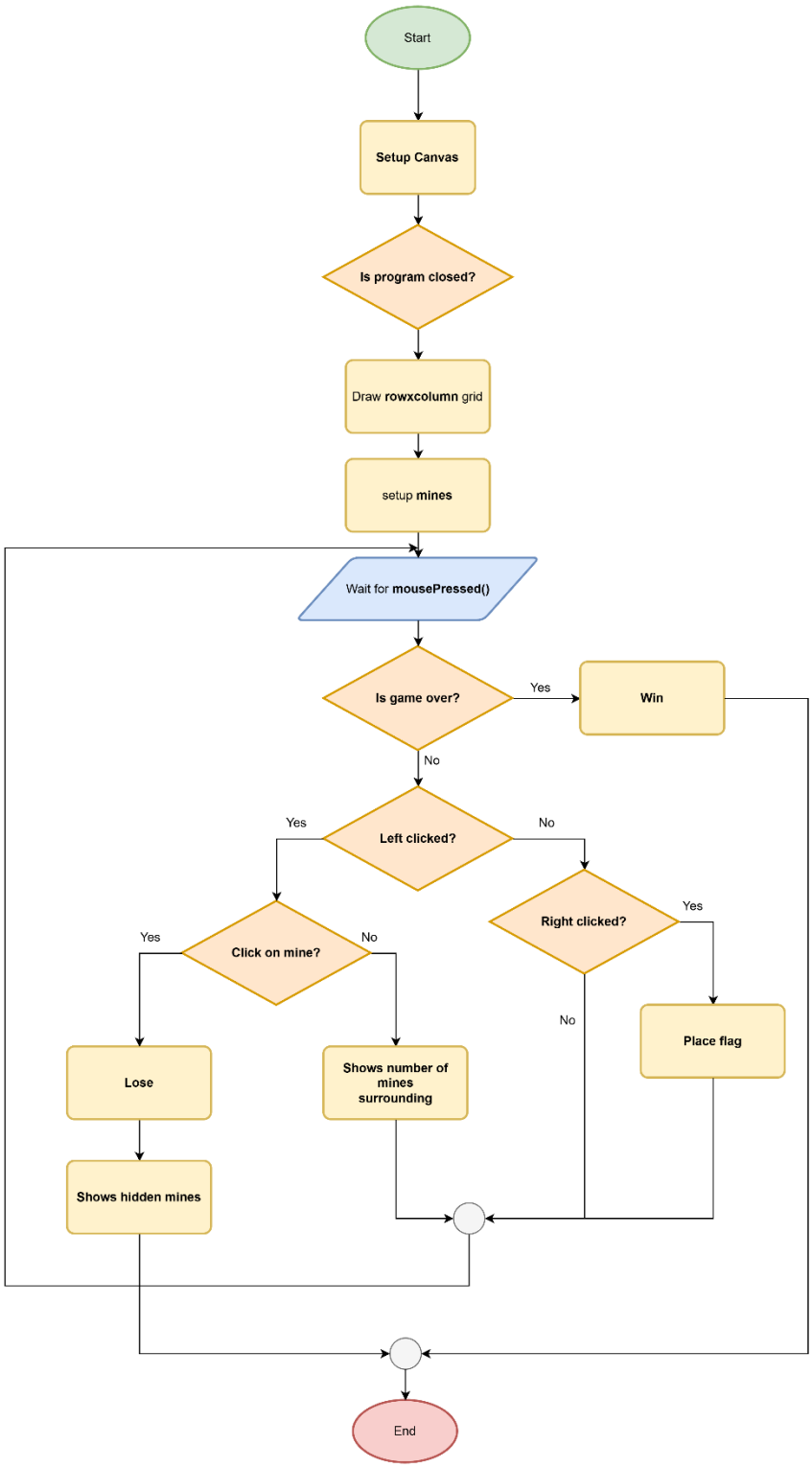
2. reveal_cell(r, c):

- หากเปิดโดนระเบิด จะตั้ง game_over = True และเปิดเผยระเบิดทั้งหมดด้วย reveal_all_mines()

- หากเปิดได้เลข 0 (ไม่มีระเบิดรอบตัว) จะใช้ Flood-fill (Recursion) เปิดช่องว่างรอบข้างทั้ง 8 ช่องให้อัตโนมัติ

3. check_win(): ตรวจสอบว่าช่องที่ไม่ใช่ระเบิดถูกเปิดครบทุกช่องแล้วหรือยัง หากครบจะตั้งค่า game_won = True

Flowchart


Start
Setup Canvas
Is program closed?
Draw rowxcolumn grid
setup mines
Wait for mousePressed()
Is game over?
Yes
Win
No
Left clicked?
Yes
No
Click on mine?
Yes
No
Lose
Shows hidden mines
Shows number of mines surrounding
Right clicked?
Yes
No
Place flag
End

Code และตัวอย่าง Input/Output

```
import random

ROWS = 10          # Number of rows
COLS = 10          # Number of columns
CELL_SIZE = 40     # Pixel width/height of each cell
NUM_MINES = 10     # Total number of hidden mines
HEADER_HEIGHT = 60 # Top HUD bar height

# Game state variables
mines = []
revealed = []
flagged = []
neighbor_counts = []
game_over = False
game_won = False
first_click = True

def settings():
    #Dynamic window size
    size(COLS * CELL_SIZE, ROWS * CELL_SIZE + HEADER_HEIGHT)

def setup():
    textAlign(CENTER, CENTER)
    reset_game()
def place_mines(first_r, first_c):
    all_cells = [(r, c) for r in range(ROWS) for c in range(COLS) if not (r == first_r and c == first_c)]

    # Ensure mines do not exceed available slots
    mine_count = min(NUM_MINES, len(all_cells))
    chosen = random.sample(all_cells, mine_count)
    for r, c in chosen:
        mines[r][c] = True

    # Precompute neighboring mine counts
    for r in range(ROWS):
        for c in range(COLS):
            if not mines[r][c]:
                count = 0
                for dr in (-1, 0, 1):
                    for dc in (-1, 0, 1):
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < ROWS and 0 <= nc < COLS and mines[nr][nc]:
                            count += 1
                neighbor_counts[r][c] = count
def draw():
    background(240)
    draw_header()
    draw_board()

def draw_header():
    fill(45, 52, 54)
    noStroke()
    rect(0, 0, width, HEADER_HEIGHT)

    flags_count = sum(row.count(True) for row in flagged)
    remaining_mines = max(0, NUM_MINES - flags_count)

    # Left: Remaining flags / mines
    fill(255)
    textSize(max(13, int(HEADER_HEIGHT * 0.28)))
    textAlign(LEFT, CENTER)
    text("Mines: {}".format(remaining_mines), 16, HEADER_HEIGHT // 2)

    # Right: Grid dimensions
    textAlign(RIGHT, CENTER)
    text("{}x{}".format(ROWS, COLS), width - 16, HEADER_HEIGHT // 2)

    # Center: Game Status
    textAlign(CENTER, CENTER)
    textSize(max(14, int(HEADER_HEIGHT * 0.32)))
    if game_over:
        fill(255, 82, 82)
        text("Game Over! (Click)", width // 2, HEADER_HEIGHT // 2)
    elif game_won:
        fill(76, 175, 80)
        text("You Win! (Click)", width // 2, HEADER_HEIGHT // 2)
    else:
        fill(255)
        text("Minesweeper", width // 2, HEADER_HEIGHT // 2)
```

```
def draw_board():
    num_colors = {
        1: color(25, 118, 210),
        2: color(56, 142, 60),
        3: color(211, 47, 47),
        4: color(123, 31, 162),
        5: color(255, 143, 0),
        6: color(0, 151, 167),
        7: color(0, 0, 0),
        8: color(128, 128, 128)
    }
    for r in range(ROWS):
        for c in range(COLS):
            x = c * CELL_SIZE
            y = r * CELL_SIZE + HEADER_HEIGHT

            stroke(170)
            strokeWeight(1)

            if revealed[r][c]:
                fill(225, 228, 232)
                rect(x, y, CELL_SIZE, CELL_SIZE)

                if mines[r][c]:
                    # Proportional mine circle
                    fill(220, 53, 69)
                    noStroke()
                    ellipse(x + CELL_SIZE * 0.5, y + CELL_SIZE * 0.5, CELL_SIZE * 0.52, CELL_SIZE * 0.52)
                elif neighbor_counts[r][c] > 0:
                    # Scaled numbers
                    val = neighbor_counts[r][c]
                    fill(num_colors.get(val, color(0)))
                    textSize(int(CELL_SIZE * 0.55))
                    textAlign(CENTER, CENTER)
                    text(str(val), x + CELL_SIZE * 0.5, y + CELL_SIZE * 0.5)
            else:
                # Hidden tile
                fill(200, 205, 210)
                rect(x, y, CELL_SIZE, CELL_SIZE)

                if flagged[r][c]:
                    # Proportional flag icon
                    fill(230, 50, 50)
                    noStroke()
                    triangle(
                        x + CELL_SIZE * 0.32, y + CELL_SIZE * 0.22,
                        x + CELL_SIZE * 0.76, y + CELL_SIZE * 0.40,
                        x + CELL_SIZE * 0.32, y + CELL_SIZE * 0.58
                    )
                    stroke(60)
                    strokeWeight(max(1.5, CELL_SIZE * 0.05))
                    line(x + CELL_SIZE * 0.32, y + CELL_SIZE * 0.20, x + CELL_SIZE * 0.32, y + CELL_SIZE * 0.80)
def mousePressed():
    global game_over, game_won, first_click
    if game_over or game_won:
        reset_game()
        return

    if mouseY < HEADER_HEIGHT:
        return
    c = mouseX // CELL_SIZE
    r = (mouseY - HEADER_HEIGHT) // CELL_SIZE
    if not (0 <= r < ROWS and 0 <= c < COLS):
        return

    if mouseButton == RIGHT:
        if not revealed[r][c]:
            flagged[r][c] = not flagged[r][c]
    elif mouseButton == LEFT:
        if flagged[r][c]:
            return
        if first_click:
            place_mines(r, c)
            first_click = False
        reveal_cell(r, c)
        check_win()
```

```
def reveal_cell(r, c):
    global game_over
    if not (0 <= r < ROWS and 0 <= c < COLS):
        return
    if revealed[r][c] or flagged[r][c]:
        return

    revealed[r][c] = True

    if mines[r][c]:
        game_over = True
        reveal_all_mines()
        return

    #Recursive flood-fill for zero-neighbor cells
    if neighbor_counts[r][c] == 0:
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                if dr != 0 or dc != 0:
                    reveal_cell(r + dr, c + dc)
```

```
def reveal_all_mines():
    for r in range(ROWS):
        for c in range(COLS):
            if mines[r][c]:
                revealed[r][c] = True

def check_win():
    global game_won
    for r in range(ROWS):
        for c in range(COLS):
            if not mines[r][c] and not revealed[r][c]:
                return
    game_won = True
```

